Лабораторная работа № 6

Тема: Классы в JavaScript: инкапсуляция и статика.

Цель: Закрепить знания о синтаксисе классов, конструкторе, геттерах/сеттерах и статических свойствах и методах.

Вариант 1. Класс Rectangle

Задание 1. Создание класса. Создать класс Rectangle с приватными полями #width и #height. Добавить конструктор, принимающий ширину и высоту. Добавить метод area(), возвращающий площадь. Создать три объекта с разными размерами и вывести площадь каждого.

Задание 2. Геттеры. Добавить в класс Rectangle геттер perimeter — вычисляет и возвращает периметр. Добавить геттер info — возвращает строку вида "Прямоугольник 4x6, площадь: 24, периметр: 20". Вызвать оба геттера для каждого объекта и вывести результат в консоль.

Задание 3. Сеттер. Добавить в класс Rectangle сеттер size — принимает массив [width, height] и обновляет приватные поля. Добавить валидацию: ширина и высота должны быть больше нуля, иначе вывести ошибку в консоль и не менять значения. Проверить сеттер с корректными и некорректными значениями.

Задание 4. Статические методы. Добавить статический метод compare(a, b) — принимает два экземпляра Rectangle, сравнивает их площади и возвращает тот, у которого площадь больше. Добавить статический метод isSquare(rect) — возвращает true если прямоугольник является квадратом. Проверить оба метода и вывести результаты в консоль.

Задание 5. Статический счётчик. Добавить в класс Rectangle статическое свойство count = 0 — счётчик созданных объектов. Увеличивать его в конструкторе при каждом создании экземпляра. Добавить статический метод getCount() — возвращает строку вида "Создано прямоугольников: 3". Создать несколько объектов и убедиться, что счётчик работает корректно.

Вариант 2. Класс Student

Задание 1. Создание класса. Создать класс Student с приватными полями #name и #year (год поступления). Добавить конструктор, принимающий имя и год поступления. Добавить метод getInfo() — возвращает строку вида "Иван Иванов, год поступления: 2022". Создать три объекта и вывести результат getInfo() для каждого.

Задание 2. Геттеры. Добавить в класс Student геттер name — возвращает имя студента. Добавить геттер course — вычисляет текущий курс студента на основе текущего года (new Date().getFullYear() - this.#year + 1). Вызвать оба геттера для каждого объекта и вывести результат в консоль.

Задание 3. Сеттер. Добавить в класс Student сеттер course — принимает номер курса и автоматически пересчитывает и обновляет #year (year = текущий год - курс + 1). Добавить валидацию: курс должен быть от 1 до 4, иначе вывести ошибку в консоль и не менять значение. Проверить сеттер с корректными и некорректными значениями.

Задание 4. Статические методы. Добавить статическое свойство university = "Колледж". Добавить статический метод isSameYear(student1, student2) — возвращает true если год поступления совпадает, иначе false. Добавить статический метод compare(student1, student2) — возвращает того студента, кто поступил позже (год поступления больше). Проверить все три и вывести результаты в консоль.

Задание 5. Статический счётчик. Добавить в класс Student статическое свойство count = 0 — счётчик созданных студентов. Увеличивать его в конструкторе при каждом создании экземпляра. Добавить статический метод getCount() — возвращает строку вида "Студентов в системе: 3". Создать несколько объектов и убедиться, что счётчик работает корректно.

📌 Шпаргалка: Классы в JavaScript

🔷 1. Определение класса

```
class Car {
  constructor(brand, model, year) {
    this.brand = brand;   // свойства
    this.model = model;
    this.year = year;
  }
```

```
  // метод экземпляра
  getInfo() {
    return `${this.brand} ${this.model} (${this.year})`;
  }
}

let car = new Car("Toyota", "Corolla", 2020);
console.log(car.getInfo()); // Toyota Corolla (2020)
```

constructor() инициализирует объект при new.

Методы пишутся внутри тела класса.

🔷 2. Геттеры и сеттеры

```
class Car {
  constructor(brand, model, year) {
    this.brand = brand;
    this.model = model;
    this.year = year;
  }

  // геттер
  get age() {
    return new Date().getFullYear() - this.year;
  }

  // сеттер
  set age(value) {
    this.year = new Date().getFullYear() - value;
  }
}

let car = new Car("BMW", "X5", 2018);
console.log(car.age);  // 7 (если 2025 год)

car.age = 3;           // переписываем "возраст"
console.log(car.year); // 2022
```

get позволяет обращаться к методу как к свойству.

set помогает контролировать изменение свойств.

🔷 3. Статические свойства и методы

```
class Car {
  constructor(brand, model) {
    this.brand = brand;
    this.model = model;
  }

  getInfo() {
    return `${this.brand} ${this.model}`;
  }

  // статическое свойство
  static type = "vehicle";

  // статический метод
  static compare(c1, c2) {
    return c1.brand.localeCompare(c2.brand);
  }
}

let c1 = new Car("Audi", "A4");
let c2 = new Car("Mercedes", "C-Class");

console.log(Car.type);              // vehicle
console.log(Car.compare(c1, c2));   // -1 (Audi < Mercedes)
```

Статические члены принадлежат классу, а не объекту.

Удобно для «общей логики» (сравнение, константы и т.д.).

🔷 4. Мини-пример

```
class Book {
  constructor(title, author, year) {
    this.title = title;
    this.author = author;
    this.year = year;
  }

  get summary() {
    return `${this.title} (${this.year}), автор:
${this.author}`;
```

```
  }

  static category = "Literature";
}

let b = new Book("1984", "George Orwell", 1949);
console.log(b.summary);         // 1984 (1949), автор:
George Orwell
console.log(Book.category);     // Literature
```